

Smart TecnoHouse

Memoria — Manual del Sistema Domótico IoT

Marco Antonio Moreno Rodríguez · Programación Orientada a Objetos · Curso 2025-26

1. Descripción del proyecto

Smart TecnoHouse es un sistema de control domótico IoT desarrollado en Java que permite monitorizar sensores del hogar y controlar actuadores de forma automática mediante reglas de negocio o de forma manual desde una interfaz gráfica. El sistema re-implementa y evoluciona el prototipo anterior desarrollado en Python, aplicando principios avanzados de Programación Orientada a Objetos.

2. Decisiones de diseño y principios SOLID

2.1 Jerarquía de clases

El diseño se basa en una jerarquía de tres niveles que aplica el principio de responsabilidad única y el principio abierto/cerrado:

- **IDispositivo (interfaz)**: contrato común para todos los dispositivos. Define los métodos `getID()`, `getNombre()` y `getEstadoActual()`, garantizando que el sistema puede tratar sensores y actuadores de forma homogénea mediante polimorfismo.
- **Sensor y Actuador (abstractas)**: clases abstractas que contienen la lógica común a todos los sensores (validación de rango, historial) y a todos los actuadores (estados posibles, cambio de estado). Ninguna de las dos conoce a la otra.
- **Clases concretas**: `SensorTemperatura`, `SensorLuz`, `SensorPresencia`, `SensorHumedad`, `ActuadorBombilla`, `ActuadorVentilador` y `ActuadorCerradura`. Cada una solo define sus valores específicos en el constructor.

Esta estructura aplica el Principio Abierto/Cerrado: añadir un dispositivo nuevo no requiere modificar ninguna clase existente, solo crear una nueva subclase.

2.2 Principios SOLID aplicados

Principios SOLID	
S — Responsabilidad única	Cada clase tiene un único motivo de cambio: Sensor gestiona lecturas, Actuador gestiona estados, Logger escribe el log, cada Regla encapsula una condición.
O — Abierto/Cerrado	Se pueden añadir nuevos sensores, actuadores o reglas sin modificar las clases existentes.
L — Sustitución de Liskov	Cualquier <code>SensorTemperatura</code> puede usarse donde se espera un <code>Sensor</code> , y cualquier <code>ActuadorBombilla</code> donde se espera un <code>Actuador</code> .
I — Segregación de interfaces	<code>IDispositivo</code> define solo lo necesario. <code>IRegla</code> define solo el método <code>aplicar()</code> .
D — Inversión de dependencias	<code>SmartTecnoHouse</code> depende de las abstracciones <code>Sensor</code> , <code>Actuador</code> e <code>IRegla</code> , no de las clases concretas.

3. Patrones de diseño

3.1 Patrón Strategy — Sistema de reglas

El patrón Strategy encapsula cada regla de negocio en su propia clase independiente. Todas implementan la interfaz `IRegla` con el método void `aplicar(List<Sensor>, List<Actuador>)`. `SmartTecnoHouse` mantiene una `List<IRegla>` y en `ejecutarReglas()` simplemente itera sobre ella llamando a `aplicar()` en cada regla, sin saber ni importarle qué hace cada una por dentro.

Las reglas implementadas son: `ReglaVentilacion` (temperatura > 28°C con presencia → ventilador HIGH), `ReglaIluminacion` (luz < 300 lux con presencia → bombilla ON), `ReglaConfort` (temperatura 20-25°C y luz > 700 lux, o casa vacía → todo OFF), y `ReglaCerradura` (sin presencia → LOCKED, con presencia → UNLOCKED).

Ventaja: añadir una regla nueva es tan simple como crear una nueva clase que implemente `IRegla`, sin tocar ninguna clase existente.

3.2 Patrón MVC — Arquitectura de la aplicación

Capas MVC	
Modelo	SmartTecnoHouse, Sensor, Actuador, IRegla y sus implementaciones. Contiene toda la lógica de negocio. No importa javax.swing en ningún momento.
Vista	VentanaPrincipal (JFrame con Swing). Solo muestra información y captura clics. No contiene lógica de negocio.
Controlador	Controlador (implements ActionListener). Escucha los eventos de la Vista, llama al Modelo y actualiza la Vista con los nuevos estados.

4. Nuevos dispositivos añadidos

- **SensorHumedad (sensor nuevo):** mide el porcentaje de humedad relativa del aire (rango 0-100%). Se conecta con el ActuadorVentilador mediante una regla de ventilación por humedad alta, demostrando la extensibilidad del sistema.
- **ActuadorCerradura (actuador nuevo):** gestiona el bloqueo electrónico de la puerta principal con estados LOCKED y UNLOCKED. La ReglaCerradura la controla automáticamente según el sensor de presencia: se bloquea cuando no hay nadie en casa y se desbloquea cuando se detecta presencia.

5. Manual de usuario

5.1 Arranque del sistema

Ejecutar la clase Main.java desde VS Code. Se abre la ventana principal con todos los sensores en estado 'Inactivo' y los actuadores en sus estados por defecto (ventilador y bombilla OFF, cerradura LOCKED).

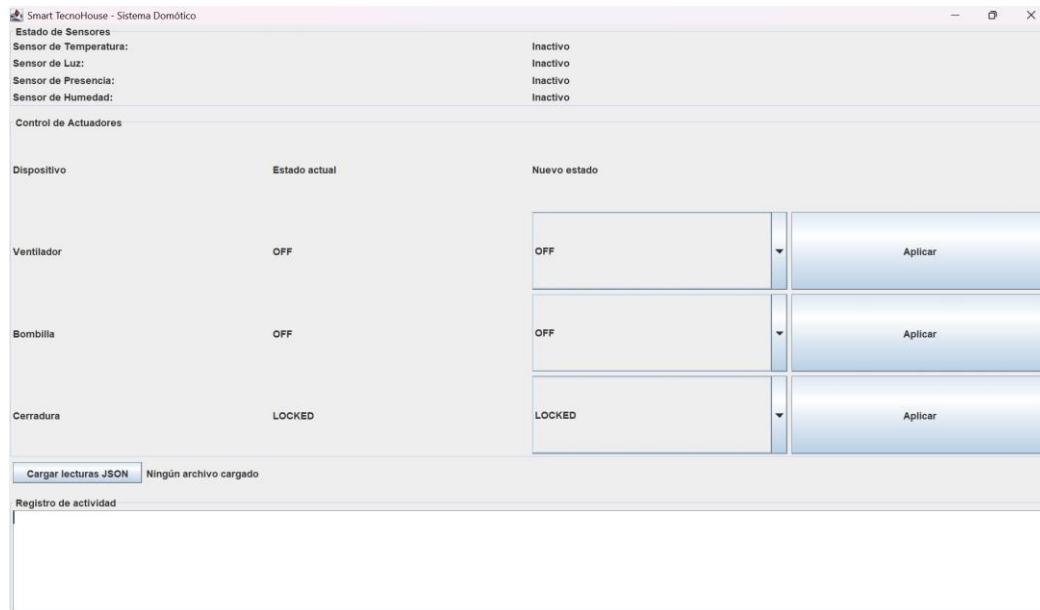


Figura 1 — Estado inicial: sensores inactivos, actuadores en reposo y cerradura bloqueada

5.2 Cargar lecturas de sensores

Pulsar el botón 'Cargar lecturas JSON' y seleccionar uno de los archivos de la carpeta data/ del proyecto. El sistema actualiza los sensores, ejecuta automáticamente todas las reglas y refleja los nuevos estados de los actuadores en pantalla. El registro de actividad muestra la hora de la carga.

Ejemplo con test_calor.json: temperatura 30°C, luz 200 lux, presencia detectada. Las reglas activan el ventilador a HIGH y encienden la bombilla, mientras la cerradura se desbloquea al detectar presencia.

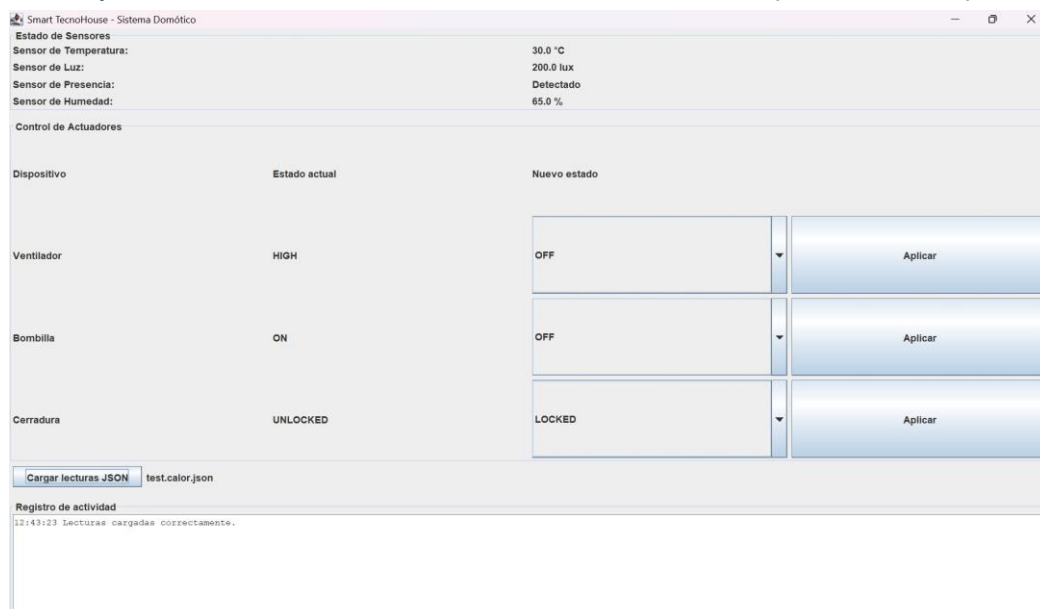


Figura 2 — Escenario de calor: ventilador HIGH, bombilla ON, cerradura UNLOCKED

5.3 Casa vacía — seguridad automática

Con test_casa_vacia.json (presencia = 0), el sistema apaga todos los dispositivos y bloquea automáticamente la cerradura, independientemente de los valores de temperatura y luz.

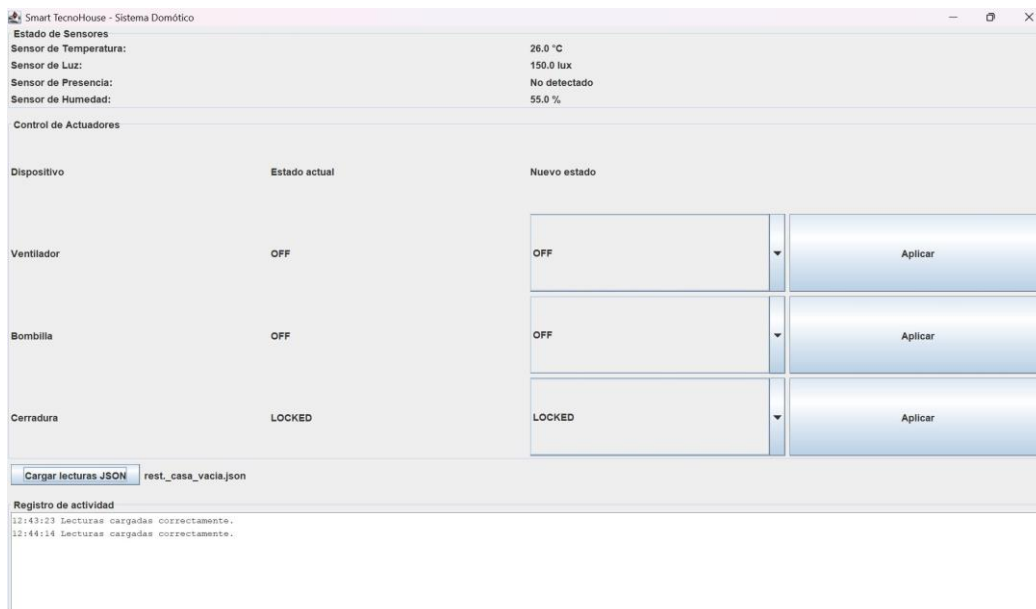


Figura 3 — Casa vacía: todo apagado y cerradura LOCKED automáticamente

5.4 Control manual de actuadores

En la sección 'Control de Actuadores', seleccionar el estado deseado en el desplegable de cualquier actuador y pulsar 'Aplicar'. El cambio se registra inmediatamente en el área de log con la hora y el origen MANUAL.

5.5 Archivos de prueba disponibles

Archivos JSON en carpeta data/	
test_calor.json	Temperatura 30°C, luz 200 lux, presencia activa. Activa ventilador HIGH y bombilla.
test_confort.json	Temperatura 22°C, luz 800 lux, presencia activa. Todo en reposo por condiciones de confort.
test_casa_vacia.json	Sin presencia. Apaga todo y bloquea la cerradura automáticamente.